RoomSense: Generalisable Occupancy Estimation from Wireless CSI

Background and Motivation

Wireless signals do more than carry data. As they propagate through an indoor environment, they are reflected, scattered, and attenuated by walls, furniture, people, and other objects. Changes in the environment can therefore alter the propagation paths observed between transmitting and receiving antennas.

Many modern wireless systems can expose these propagation characteristics through **Channel State Information (CSI)**. CSI provides a fine-grained description of how the wireless channel affects transmitted signals across different frequency components and antennas. Rather than directly observing the environment, CSI provides a time-varying representation of its radio propagation conditions.

This makes CSI useful for a broader class of **wireless sensing** problems, in which properties of an environment are inferred from how they affect radio signals. One such property is **occupancy**: the number of people present in an indoor space.

Reliable occupancy estimation could support applications such as adaptive building operation, energy management, space utilisation, and safety systems. Unlike image-based sensing, CSI does not directly capture visual appearance. In this challenge, the objective is specifically limited to estimating **aggregate occupancy count**; it does not involve identifying, localising, or tracking individuals.

The challenge, however, is not simply to detect changes in a wireless channel. Indoor propagation is strongly affected by room geometry, furniture, construction materials, device placement, and surrounding radio conditions. A model can therefore appear accurate because it has learned the characteristic propagation patterns of the environments represented in its training data rather than a relationship that transfers to new environments.

The more interesting problem is consequently one of **generalisation**: can a model learn occupancy-related structure in CSI that remains useful when the physical environment changes?

This challenge focuses on that harder and more practically meaningful objective: **estimating indoor occupancy from wireless CSI while generalising to previously unseen rooms**.

Problem Overview

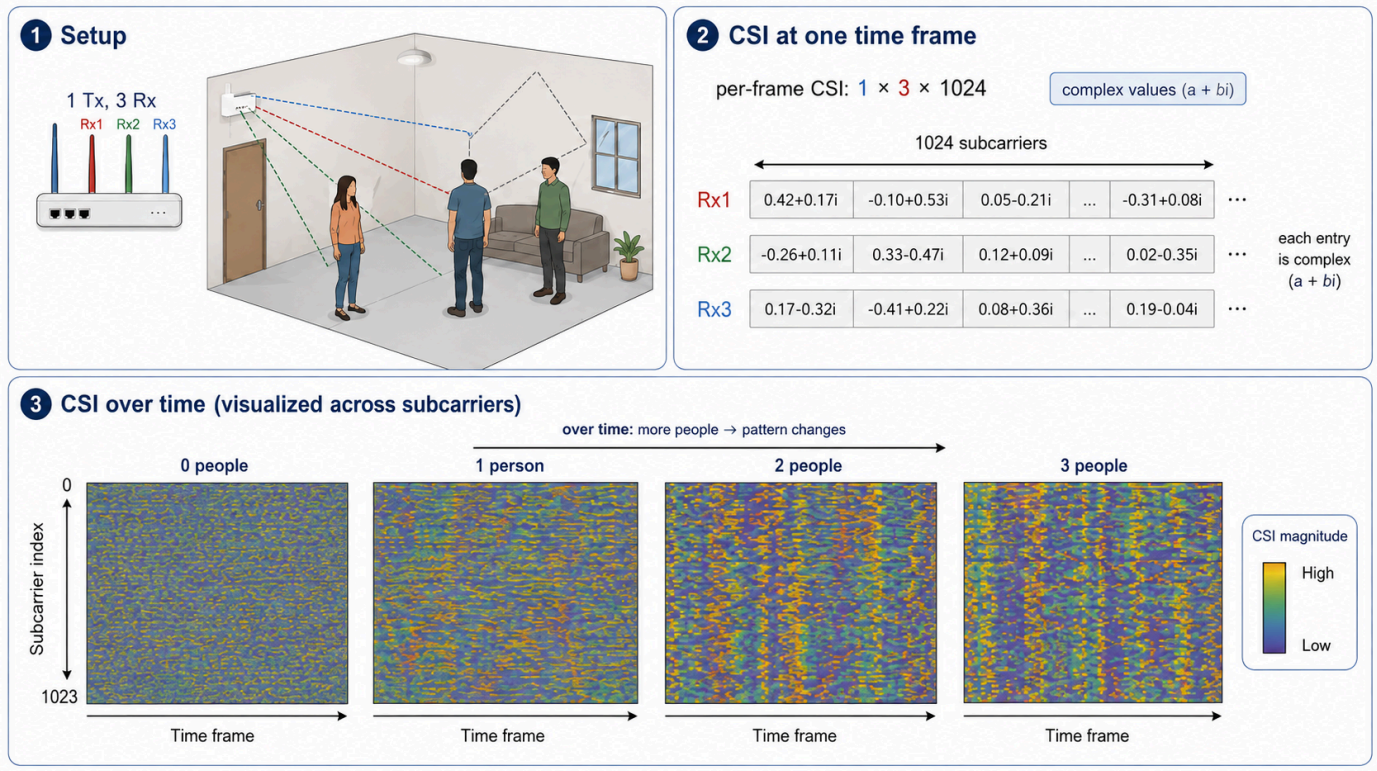
The goal of the challenge is to develop a model that uses CSI measurements to estimate the number of people present in an indoor environment.

The measurements in this challenge are obtained from commercial WiFi devices in controlled testbed environements with informed volunteers, but the underlying problem is more general: learning environmental information from the structure and temporal variation of a wireless propagation channel.

Each sample contains CSI observations collected between transmitting and receiving antennas over a short period of time. CSI describes how the propagation channel affects many narrow frequency components, or **subcarriers**, of the transmitted signal.

Human presence contributes to this channel because the human body can reflect, scatter, attenuate, and obstruct radio propagation. As the number, position, posture, and movement of people vary, the resulting multipath channel can change across antennas, subcarriers, and time.

These effects are indirect. CSI does not provide an image or explicit representation of the people in a room, and there is no simple one-to-one mapping between a single channel observation and an occupancy count. The same occupancy may produce substantially different CSI in different environments, while changes unrelated to occupancy may also alter the measured channel.



A central requirement of the challenge is therefore **cross-room generalisation**. The rooms used for evaluation are different from those used for training. A successful model cannot rely solely on memorising the background channel, device placement, or other characteristic signal patterns of individual training rooms. Instead, it must extract information that remains predictive of occupancy when these environmental conditions change.

Evaluation environments may share broader characteristics or layouts with environments represented in the training data, but the specific rooms and measurement samples are unseen. For some environmental configurations, only a limited amount of labelled data may be available. Models must

therefore make effective use of limited supervision while transferring useful structure learned from other rooms and conditions.

Contestants are encouraged to consider methods that:

- distinguish occupancy-related variation from environment-specific channel characteristics;
- exploit structure shared across rooms and propagation conditions;
- learn effectively from limited labelled CSI measurements;
- make use of temporal as well as frequency- and antenna-domain information; and
- remain robust when applied to environments not encountered during training.

The challenge therefore asks a broader question than whether occupancy can be predicted in a known room:

Can a model learn transferable information about occupancy from wireless channel measurements, even when the propagation environment itself is unfamiliar?

Concepts

The samples in our dataset are acquired by stationary Wi-Fi routers in rooms, where each room is equipped with a single router. Multiple **sample**s can be acquired in the same **room**, and multiple rooms can share the same **layout** (i.e. floor plan). Every router is identical, and routers in rooms sharing the same layout are identically deployed.

Formally, the concepts are represented as follows:

- For each sample x, define $\mathcal{R}(x)$ as the room in which this sample is acquired.
- For each sample set X, define $\mathcal{R}(X)$ as the rooms in which these samples are acquired, i.e. $\{\mathcal{R}(x) \mid x \in X\}$.
- For each room y, define $\mathcal{L}(y)$ as the layout that this room follows.
- For each room set Y, define $\mathcal{L}(Y)$ as the layouts that these rooms follow, i.e. $\{\mathcal{L}(y) \mid y \in Y\}$.
- For each sample set X, define $\mathcal{L}(X)$ as the layouts that the rooms in which these samples are acquired follow, i.e. $\mathcal{L}(\mathcal{R}(X))$.

Dataset

The dataset is designed to evaluate **cross-room generalisation**: the ability to predict headcount in rooms for which no labelled training data are provided. While information about the radio environment may still be learned from the unlabelled training data, the model must transfer the relationship between CSI observations and headcount from labelled data collected in other rooms.

The dataset is composed of three sets:

- **Labelled training set (S):** samples with ground-truth labels indicating the number of people present.
- **Unlabelled training set (U):** raw samples without headcount labels. These data may be used for unsupervised or self-supervised learning and may contain observations from a broader set of rooms than the labelled training set.
- **Test set (T):** labelled samples withheld for evaluation. Test samples are collected from rooms that do not appear in the labelled training set, such that headcount prediction requires generalisation beyond the rooms for which supervision is available. However, unlabelled observations from the same or related environments may be present in UU.

At the **sample level**, the three sets are mutually disjoint. At the **room level**, the defining cross-room constraint is that no test room appears in the labelled training set, while overlap between rooms represented in the unlabelled training set and the test set is permitted.

Formally, $\mathcal{R}(X)$ being the set of physical rooms represented in dataset X:

$\mathcal{R}(S) \cap \mathcal{R}(T) = \varnothing$, while

$\mathcal{R}(U) \cap \mathcal{R}(T)$ is not required to be empty.

The layouts represented in the test set are drawn from the family of layouts represented in the training data, so that the primary challenge is **generalisation across physical rooms rather than extrapolation to entirely new layout types**.

Evaluation

This problem is formulated as a **(P+1)-class classification task**, where each possible headcount from **0 to (P)**, inclusive, is treated as a distinct class. The value of (P), representing the maximum supported occupancy, is specified when the datasets are released.

The task is evaluated as classification rather than regression because the objective is to predict the **exact number of people present** in the room.

Submissions are scored using **classification accuracy**, defined as the proportion of test samples for which the predicted headcount exactly matches the ground-truth label.

The test set is divided into two layout-disjoint portions:

- **Public test set:** The input samples and labels remain on the platform. For each submission, contestants provide only their model. The platform computes and scores its predictions automatically, and the resulting score is displayed on the live leaderboard shortly after submission.

- **Private test set:** The samples and labels remain hidden from contestants. The resulting score against this set will only be released occasionally. The final choosen model of each team is executed once on the private test set, and its score is used for the final leaderboard.

The public and private test sets follows a same data distribution.

Formats and Submissions

Dataset Format

Contestants will have access to a folder containing the **training dataset**. Each trace corresponds to a continuous CSI measurement captured in a single room during a period in which the **number of people present remains constant**.

The training files use lossless Blosc2 compression. Extract the labelled and unlabelled archives into separate folders:

```
labeled/
  samples.csv
  CSI_<identifier>.b2
  ATI_<identifier>.b2
unlabeled/
  samples.csv
  CSI_<identifier>.b2
  ATI_<identifier>.b2
```

Each `samples.csv` index contains `identifier`, `layout`, `room`, `acquisition`, and `headcount`. Use `identifier` to locate the matching files. Unlabelled rows have `headcount` set to `unlabeled`. Download links are provided in `data_downloads.md`.

Load both arrays with `blosc2.load_array`. Their shapes, data types, and values are preserved:

```
import blosc2

csi = blosc2.load_array("CSI_<identifier>.b2")
ati = blosc2.load_array("ATI_<identifier>.b2")
```

The CSI and ATI files follow the naming conventions:

`CSI_[room_layout]_[room_id]_[time_hash]_[label].b2`

`ATI_[room_layout]_[room_id]_[time_hash]_[label].b2`

where:

- `[room_layout]` identifies the physical room layout;
- `[room_id]` identifies the individual room;
- `[time_hash]` uniquely identifies the acquisition;
- `[label]` indicates the number of people present in the room for a labelled trace. For an unlabelled trace, this field is set to `unlabeled`.

Do not be surprised if `[room_layout]` and/or `[room_id]` is set to `unknown`. These traces were collected under less controlled field conditions, where the corresponding metadata may not be available. In particular, **two traces marked `unknown` should not be assumed to originate from the same room or the same room layout**.

Trace Length and Sampling

Each training trace contains **12,000 consecutive CSI frames**. Validation and test samples are shorter and contain exactly **600 consecutive frames**.

The nominal sampling rate is approximately **200 Hz** for both training and validation/test data. Therefore:

- training trace: 12,000 frames, approximately 60 seconds;
- validation/test sample: 600 frames, approximately 3 seconds.

For submission, models are only required to support the fixed **600-frame input length** used for validation and testing.

The occupancy is restricted to **0-4 people**, giving five possible classes. Within every validation/test sample:

- the number of people in the room remains constant throughout the 600 frames; and
- there are no people passing immediately outside the room during the acquisition.

Data Representation

More specifically, each training trace contains the following information.

CSI

`CSI.shape = (12000, 1, 3, 1024)`

`CSI.dtype = np.complex64`

The four dimensions correspond to:

`(time, Tx antenna, Rx antenna, subcarrier)`

That is, for each of the 12,000 consecutive acquisition instants, the trace contains CSI measurements between **1 transmitting antenna and 3 receiving antennas over 1,024 subcarriers**. The channel bandwidth is **80 MHz**.

ATI - Acquisition Time Interval

`ATI.shape = (12000,)`

`ATI.dtype = np.float32`

`ATI[i]` gives the elapsed time, in **microseconds (µs)**, between CSI frame `i` and CSI frame `i+1`.

Since the final frame has no subsequent frame, the last ATI value is padded with `0` so that the ATI array has the same temporal length as the CSI sequence.

Although the nominal CSI sampling rate is approximately 200 Hz, the intervals between consecutive frames are **not necessarily uniform**. For example, CSI acquisition may be delayed when the wireless medium is occupied by an ongoing transmission. This timing information is therefore provided explicitly and may be useful for occupancy inference.

Model Submission

Contestants should submit **only the trained model**, exported in **ONNX format**. Any filename ending in `.onnx` is accepted; `model.onnx` is an example. The file must be smaller than 100,000,000 bytes (100 MB), with all weights embedded inside it. The platform executes the model on CPU using ONNX Runtime.

Memory limit: 6 GB. **Prediction time:** Less than 0.7 s per sample on CPU, on average across the evaluation set.

The submitted model must expose the following input and output interface.

Inputs

Input Name	Tensor Shape	Element Type	Description
`frames`	`(600, 1, 3, 1024, 2)`	`float32`	600 consecutive CSI frames. The original complex-valued CSI is represented by two real-valued components in the final dimension for ONNX compatibility.
`intervals`	`(600,)`	`float32`	Acquisition time intervals in µs. `intervals[i]` is the elapsed time between frame `i` and frame `i+1`; the final value is padded with `0`.

The `frames` tensor therefore follows the dimension order:

`(time, Tx antenna, Rx antenna, subcarrier, complex component)`

where the final dimension of size 2 represents the real and imaginary components of the original complex CSI values.

Output

Output Name	Tensor Shape	Element Type	Description
`logits`	`(5,)`	`float32`	Classification logits corresponding, in order, to occupancy classes `0, 1, 2, 3, 4`.

The predicted headcount is obtained as:

`prediction = argmax(logits)`

Thus, output index `i` corresponds directly to a prediction of `i` **people present in the room**.